

PROJECT WORK

Analisi e Sperimentazione di un sistema di trasmissione a 2.4 GHz

Simone Dama

29/06/2026

Indice

Indice.....	2
1. Introduzione.....	3
1.1 Il flusso di lavoro.....	3
2. Contenuto del progetto.....	6
2.1 Analisi della documentazione.....	6
2.1.1 Interfaccia con la scheda.....	8
2.1.2 Funzionalità della scheda.....	9
2.1.3 Struttura dei dati.....	12
2.1.4 Radiazione dell'antenna.....	14
2.2 Test preliminari.....	16
2.2.1 Modalità di test.....	16
2.2.2 Trasmissione e ricezione dei dati.....	18
2.2.3 I dati ottenuti.....	20
2.3 Scrittura del codice per test più approfonditi.....	22
2.3.1 Flusso del programma.....	23
2.3.2 Configurazione del microcontrollore.....	26
2.3.3 Collegamento tra le schede.....	28
2.3.4 Programmazione secondo le linee guida.....	30
2.3.5 Modalità di test.....	32
2.3.6 I dati ottenuti.....	34
3. Riflessioni finali e conclusioni.....	35
4. Glossario.....	36
5. Bibliografia e sitografia.....	40

1. Introduzione

1.1 Il flusso di lavoro

Il progetto a cui ho potuto partecipare mi ha permesso di contribuire ad un'iniziativa più ampia volta ad effettuare **ricerca e sviluppo su nuove tecnologie** a fronte della richiesta di alcuni clienti. L'**obiettivo** è trovare il modo di aumentare il flusso dei dati da inviare via radio, passando da valori come 1 dato ogni 5 minuti a centinaia di dati al secondo.

In particolare, la **principale soluzione proposta** è quella di valutare ed integrare l'uso di **ricetrasmittitori a 2.4 GHz**. È stata utilizzata la **scheda Thyone-I di Wurth** per trasmettere grandi quantità di dati, situazione in cui le schede a 868 MHz possono risultare meno efficaci a causa dei limiti di banda.

In generale il **flusso di lavoro** ([Fig. 1](#)) per questo tipo di progetti si articola in **tre fasi** fondamentali:

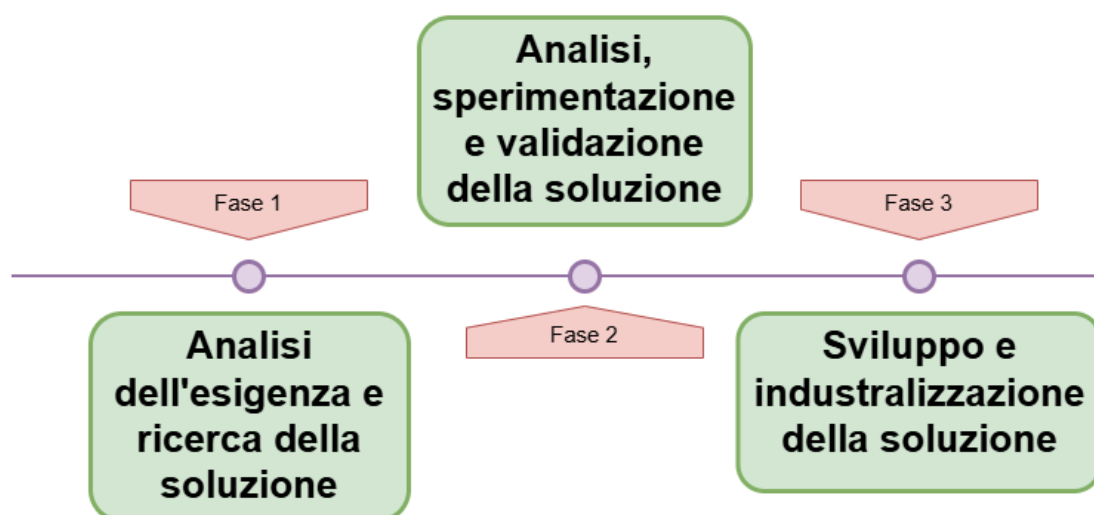


Fig. 1 Il flusso del lavoro rappresentato nelle sue tre fasi.

- **Fase 1:** viene eseguita un'attenta **analisi dell'esigenza e ricerca della soluzione** sulla scheda che è stata utilizzata. Deve essere fatta al meglio in modo tale da incontrare meno ostacoli possibili durante il percorso ed è estremamente necessaria per lo sviluppo di un'integrazione.
- **Fase 2:** è l'**analisi, sperimentazione e validazione della soluzione**: consiste nella raccolta dei dati tramite sperimentazione sulla scheda, essenziale per individuare le principali criticità della tecnologia sotto esame.
- **Fase 3:** è lo **sviluppo e industrializzazione della soluzione** e serve a portare la soluzione ad un livello commerciale.

La fase 1 è stata già svolta prima del mio arrivo, io mi sono occupato di proseguire il percorso con la **fase 2**, occupandomi di svolgere l'**analisi, la sperimentazione e la validazione della soluzione** ([Par. 2](#)), a sua volta suddivisa in altre **tre fasi** ([Fig. 2](#)).

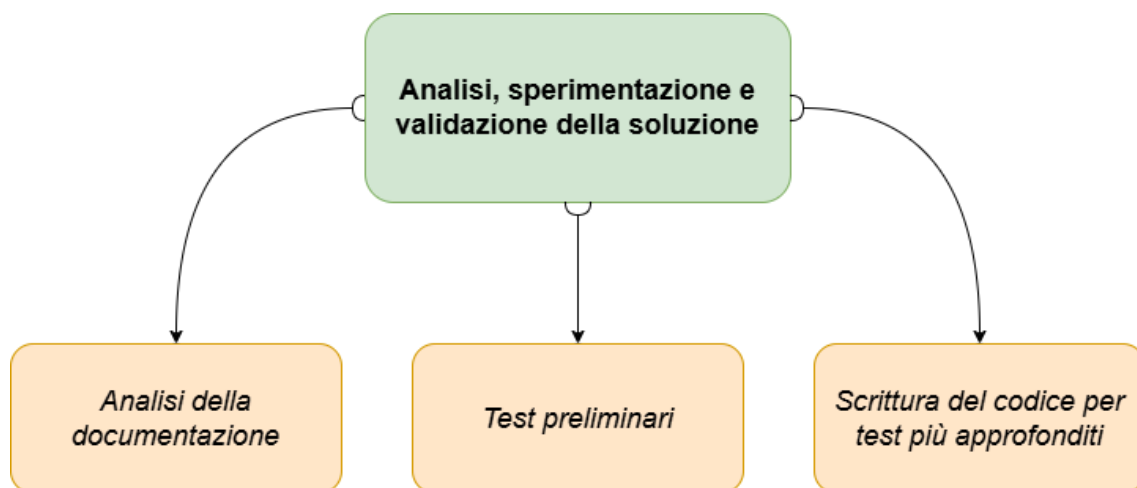


Fig. 2 *Analisi, sperimentazione e validazione della soluzione rappresentata nelle sue tre fasi.*

- **Analisi della documentazione** ([Par. 2.1](#)): studio approfondito dei datasheet della scheda a 2.4 GHz sotto esame per comprendere l'interfaccia UART, i comandi disponibili, la struttura dei dati da trasmettere/ricevere e le caratteristiche di radiazione delle antenne, al fine di pianificare correttamente i test successivi.
- **Test preliminari** ([Par. 2.2](#)): esecuzione di prove in ambienti interni ed esterni, variando la velocità di trasmissione e osservando il comportamento della scheda in presenza di ostacoli. Raccolta e analisi di parametri quali RSSI e percentuale di pacchetti persi, per individuare i reali limiti operativi della tecnologia a 2.4 GHz.
- **Scrittura del codice per test più approfonditi** ([Par. 2.3](#)): scrittura del programma in C per interfacciare la scheda Thyone-I al microcontrollore STM32 via UART nel rispetto delle regole MISRA C.

2. Contenuto del progetto

2.1 Analisi della documentazione

Prima di qualsiasi attività sperimentale, è stato necessario condurre uno **studio approfondito della documentazione tecnica** fornita dal produttore. L'obiettivo di questa fase è stato duplice: da un lato, comprendere in modo completo le caratteristiche hardware e le funzionalità della scheda Thyone-I; dall'altro, acquisire tutte le informazioni indispensabili per **pianificare correttamente i test successivi** ed **evitare errori legati ad una conoscenza superficiale del dispositivo**.

La scheda Thyone-I è disponibile in due varianti, che si differenziano per il tipo di antenna:

- Thyone-I con antenna interna in PCB ([Fig. 3](#))
- Thyone-I con antenna esterna ([Fig. 4](#))

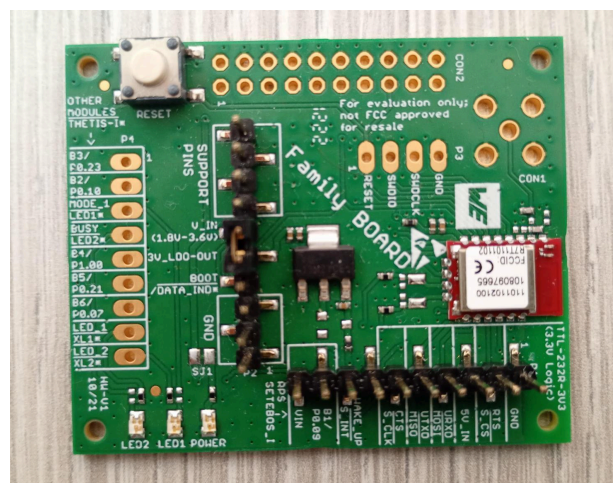


Fig. 3 La scheda radio a 2.4 GHz Thyone-I di Wurth nella sua versione con antenna interna.

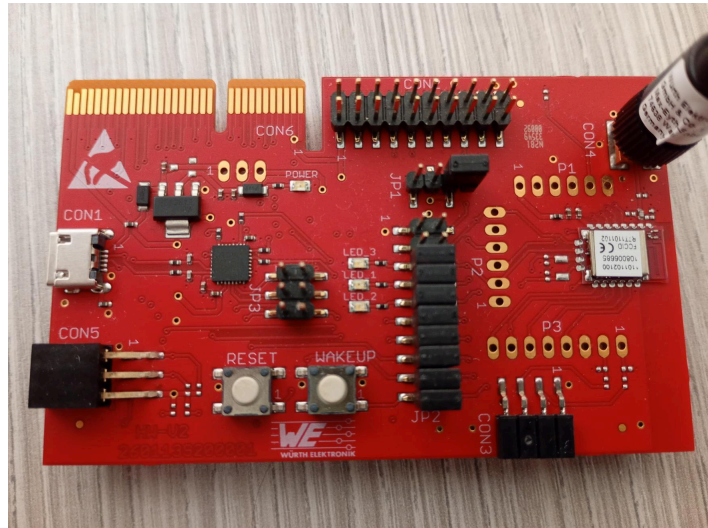


Fig. 4 La scheda radio a 2.4 GHz Thyone-I di Würth nella sua versione con antenna esterna.

2.1.1 Interfaccia con la scheda

Fondamentale è l'**interfaccia USB - TTL**: la scheda ha una logica di funzionamento per la **comunicazione UART** a 3.3V, per questo dobbiamo utilizzare un adattatore USB - TTL ([Fig. 5](#)) per configurare i parametri e testare la scheda per la prima volta.



Fig. 5 Il convertitore USB - TTL RS-232 per la comunicazione UART a 3.3 V con la scheda Thyone-I.

Questo genere di convertitori serve a far comunicare il computer con i dispositivi UART con l'ausilio di software generici per la comunicazione su terminale, ma la scheda Thyone-I che andrò ad utilizzare è supportata nativamente dal **software WE UART Terminal** ([Fig. 6](#)).

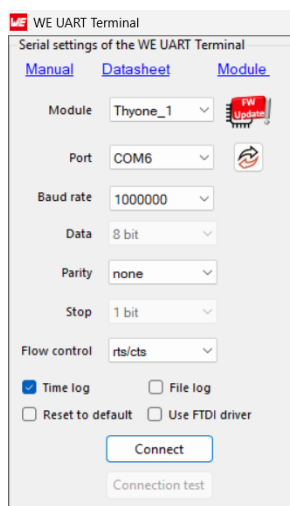
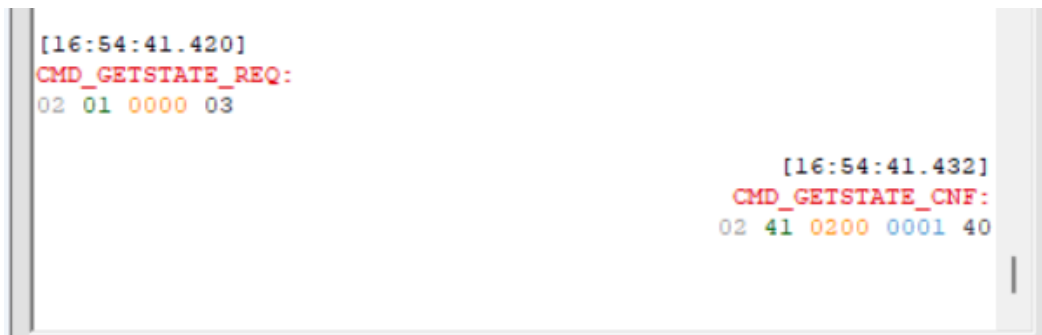


Fig. 6 Il software WE UART Terminal per la connessione UART con la scheda Thyone-I, permette di selezionare il modello, la porta seriale, il baud rate e altre impostazioni di comunicazione come il controllo del flusso e la parità.

2.1.2 Funzionalità della scheda

La scheda Thyone-I mette a disposizione un insieme di **comandi dedicati** che ne abilitano le principali funzionalità operative. Ciascun comando viene inviato tramite interfaccia seriale UART e consente di configurare, controllare o acquisire informazioni dallo stato della scheda. Di seguito vengono descritti i comandi supportati.

- **STATE:** Restituisce lo **stato attuale** della scheda ed è utile per verificare se la connessione seriale UART con il dispositivo è stata stabilita correttamente ([Fig. 7](#)).



```
[16:54:41.420]
CMD_GETSTATE_REQ:
02 01 0000 03

[16:54:41.432]
CMD_GETSTATE_CNF:
02 41 0200 0001 40
```

Fig. 7 Esempio di comunicazione utilizzando il software WE UART Terminal: invio del comando per ottenere lo stato della scheda e rispettiva risposta della scheda.

- **BROADCAST:** Permette di **inviare un payload** (dati) a tutti i moduli raggiungibili. Dopo l’invio, la scheda fornisce prima una conferma di avvenuta elaborazione del dato, poi una conferma di trasmissione avvenuta.
- **MULTICAST:** Simile al comando BROADCAST, ma **l’invio è limitato a un gruppo di moduli** che condividono lo stesso GROUP ID (configurabile). Solo i moduli con GROUP ID corrispondente riceveranno i dati.
- **UNICAST:** **Invia dati esclusivamente a un modulo con uno specifico indirizzo** MAC di destinazione. L’indirizzo MAC

deve essere configurato in un apposito registro di memoria della scheda.

- **SET CHANNEL:** Seleziona il **canale di trasmissione**, corrispondente a una determinata frequenza. Affinché due o più schede possano comunicare tra loro, devono essere configurate sullo stesso canale.
- **TX POWER:** Imposta la **potenza del segnale** radio, espressa in dBm. Ad esempio, per la scheda Thyone-I il valore massimo è di 8 dBm.
- **PROFILO RF:** Definisce il profilo che determina una diversa **velocità di trasmissione radio** dei dati ([Fig. 8](#)).

Radio profile	Description
0	125 kbit/s long range mode.
1	500 kbit/s mode.
2	1000 kbit/s mode.
3	2000 kbit/s mode.

Fig. 8 In tabella i profili radio (Profilo RF) messi a disposizione dalla scheda Thyone-I, sono corrispondenti ad una velocità di trasmissione dei dati in kbit/s.

- **UART CONFIG:** Imposta la **velocità di trasmissione dei dati sulla porta UART**, misurata in baud. Per aumentare l'affidabilità della comunicazione, è possibile integrare il **controllo del flusso** (flow control), che utilizza due pin aggiuntivi: RTS (Ready To Send) e CTS (Clear To Send). Questi pin segnalano quando una scheda è pronta per inviare o ricevere dati.
- **MODULE MODE:** Seleziona la **modalità operativa** principale della scheda: normale oppure ripetitore ([Fig. 9](#)). In **modalità ripetitore**, la scheda ritrasmette i segnali ricevuti da altri moduli, estendendo la copertura all'interno di una **rete mesh**.

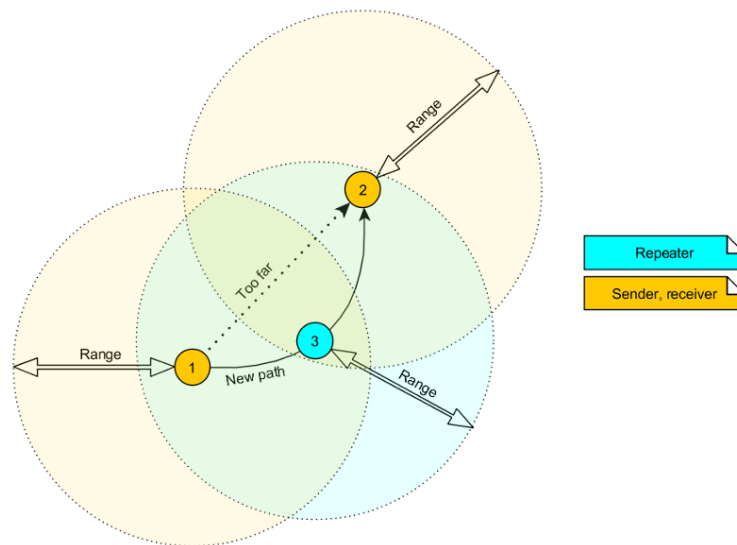


Fig. 9 L'immagine rappresenta una rete mesh utilizzata tramite la funzione ripetitore della scheda Thyone-I.

- **SLEEP:** Porta la scheda in uno stato di ibernazione (basso consumo). La scheda può essere riattivata tramite l'apposito pin WAKE UP.

2.1.3 Struttura dei dati

Per **trasmettere** correttamente un pacchetto via radio con la scheda Thyone-I, è necessario rispettare la **struttura dei dati** ([Fig. 10](#)).

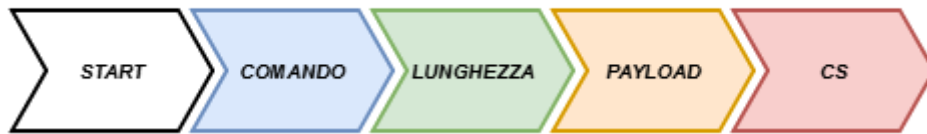


Fig. 10 Rappresentazione grafica della struttura dei dati richiesta per trasmettere un pacchetto via radio con la scheda Thyone-I.

Per **ricevere** la stessa informazione, la struttura è presentata con un formato leggermente diverso ([Fig. 11](#)).



Fig. 11 Rappresentazione grafica della struttura dei dati ricevuti via radio con la scheda Thyone-I.

Nella comunicazione radio tra moduli Thyone-I, sia in trasmissione che in ricezione, ogni messaggio è composto dai seguenti campi:

- Il byte di **start** indica l'inizio del messaggio.
- Il **comando** identifica ciò a cui il messaggio che segue fa riferimento, in questo caso la trasmissione/ricezione di un messaggio via radio.
- La **lunghezza** del payload indica il numero di byte che compongono il payload. In ricezione è utile per sapere quanti byte attendere prima di concludere la lettura.
- Il **payload** rappresenta il messaggio vero e proprio, che contiene l'informazione da condividere.

- Il **CS (Checksum XOR)** è un controllo che viene effettuato per verificare la validità del messaggio. È un modo semplice per sapere se i dati ricevuti sono coerenti con quelli originariamente trasmessi. Viene calcolato sull'intero pacchetto, escluso il byte di CS.

In fase di ricezione sono presenti altri due campi aggiuntivi:

- L'**indirizzo** del mittente corrisponde al codice identificativo della scheda che ha inviato il messaggio.
- Il valore di **RSSI** indica l'intensità del segnale ricevuto. Un valore più alto corrisponde a una migliore qualità del segnale, il che generalmente consente velocità di trasmissione più elevate e riduce la percentuale di pacchetti persi.

2.1.4 Radiazione dell'antenna

Come mostrato in precedenza ([Par. 2.1](#)), la scheda Thyone-I è disponibile in due diverse varianti.

La differenza più evidente tra di esse è che la versione con antenna esterna è generalmente più efficiente in termini di guadagno e portata. Tuttavia, non si tratta solo di efficienza: l'antenna interna irradia il segnale nell'aria con un **diagramma di radiazione** diverso rispetto a quella esterna.

Per questo motivo, prima di effettuare qualsiasi test, è indispensabile **consultare i grafici di radiazione delle antenne** ([Fig. 12](#)). Tale lettura è importante perché permette di conoscere la **direzionalità**, consentendo di posizionare correttamente i moduli ed **evitare problemi di attenuazione dovuti all'orientamento** tra le schede ([Fig. 13](#)).

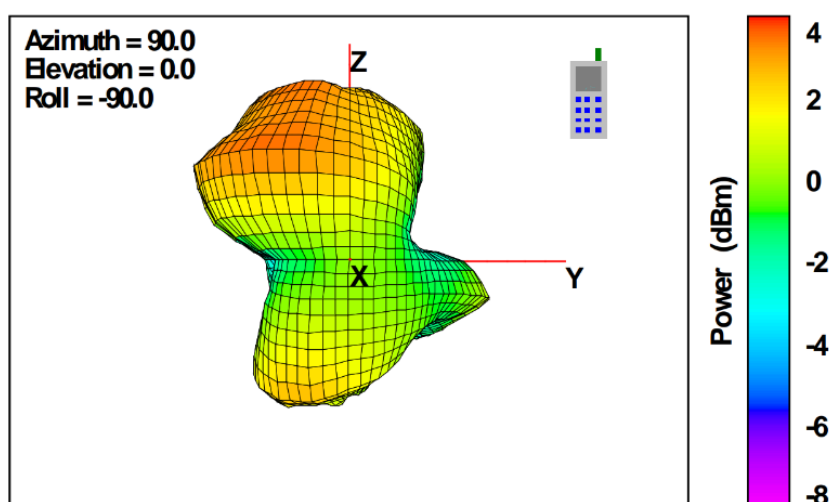


Fig. 12 Grafico di radiazione che indica i maggiori punti di emissione della scheda Thyone-I con antenna interna PCB.

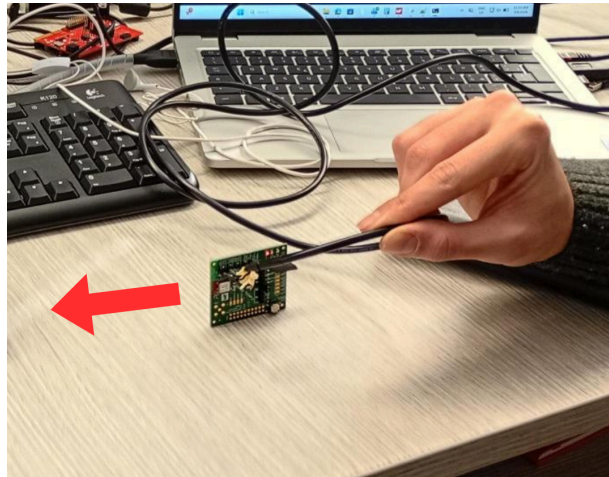


Fig. 13 Nell'immagine è rappresentata la scheda Thyone-I e la direzione di maggiore emissione dell'antenna interna.

2.2 Test preliminari

2.2.1 Modalità di test

I test sono stati condotti per **valutare le prestazioni radio** della scheda in **condizioni reali**, verificando la qualità del collegamento al variare di:

- direzione di massima emissione (antenna interna ed esterna)
- velocità di trasmissione (bassa velocità e velocità medio-alta)
- ambiente (interno ed esterno, con ostacoli e senza ostacoli)

Ciò permette di raccogliere dati utili a **individuare le principali criticità di portata, velocità di trasmissione e stabilità**.

I test sono stati condotti in entrambe le direzioni, sia dalla scheda con antenna interna ([Fig. 3](#)) verso quella con antenna esterna ([Fig. 4](#)), sia nel verso opposto.

Vengono utilizzati due valori diversi di **profilo RF** ([Fig. 8](#)) selezionabili all'interno del software: il valore 0 a 128 kbit/s (bassa velocità di trasmissione e di ricezione) utilizzato specialmente per le comunicazioni a lungo raggio, e il valore 2 a 1000 kbit/s (medio-alta velocità di trasmissione e di ricezione).

Io e il mio collega abbiamo eseguito queste prove **all'interno dell'azienda** in diverse stanze dell'edificio **con la presenza di ostacoli** ([Fig. 14](#)) ed **anche all'esterno** in un parco con diverse distanze e **con assenza di ostacoli** ([Fig. 15](#)).

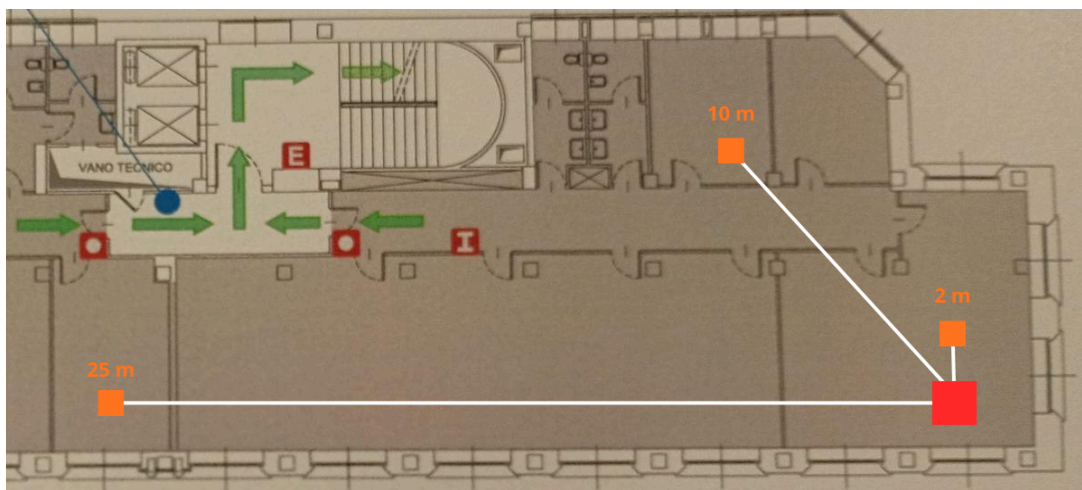


Fig. 14 L'immagine di riferimento rappresenta la piantina dell'edificio e le diverse posizioni in cui sono stati eseguiti i test preliminari. Il quadrato rosso indica la scheda con antenna esterna, mentre i quadrati arancioni indicano la scheda con antenna interna con le relative distanze dalla scheda con antenna esterna.



Fig. 15 L'immagine di riferimento rappresenta le diverse posizioni in cui sono stati eseguiti i test preliminari al Parco Paradiso a Torino. I quadrati rossi indicano la scheda con antenna esterna, mentre i quadrati arancioni indicano la scheda con antenna interna con le relative distanze dalla scheda con antenna esterna.

2.2.2 Trasmissione e ricezione dei dati

Per trasmettere i dati ho realizzato un semplice **programma in Python** che invia, ogni 2 secondi, il payload massimo consentito dalla scheda Thyone-I (224 byte). **Il programma tiene traccia dei dati inviati tramite un contatore**, in modo da consentire al ricevitore di verificare se vi sono pacchetti mancanti: l'assenza di numeri consecutivi indica infatti la perdita di uno o più pacchetti.

Ogni test viene eseguito in diverse modalità, ciascuna della durata di circa 40 secondi. Poiché un messaggio viene inviato ogni 2 secondi, per ogni modalità ci si attendono circa 20 messaggi, per un totale di circa 20×224 byte di dati attesi.

Tra i dati ricevuti via radio si ottiene principalmente il **payload** (il messaggio), il **valore di RSSI** (l'indicatore della potenza del segnale ricevuto) ed il **checksum**.

Per eseguire numerosi test è necessario **estrarre questi valori in modo rapido**, per farlo si utilizza un programma realizzato appositamente in Python, che **ottiene il valore di RSSI e verifica che i dati ricevuti, compreso il payload, siano conformi**.

Il programma salva i dati in un **file di log** in cui vengono inserite tutte le informazioni relative ai test effettuati con il relativo timestamp indicante ora, minuti e secondi in cui è avvenuta la ricezione del dato.

Durante la fase di test preliminari sul campo le schede sono connesse tramite UART con convertitore USB - TTL RS-232 ([Fig. 16, 17](#)).



Fig. 16 Collegamento della scheda Thyone-I con antenna interna durante la fase di test preliminari sul campo, viene utilizzato il convertitore USB-to-TTL RS-232.

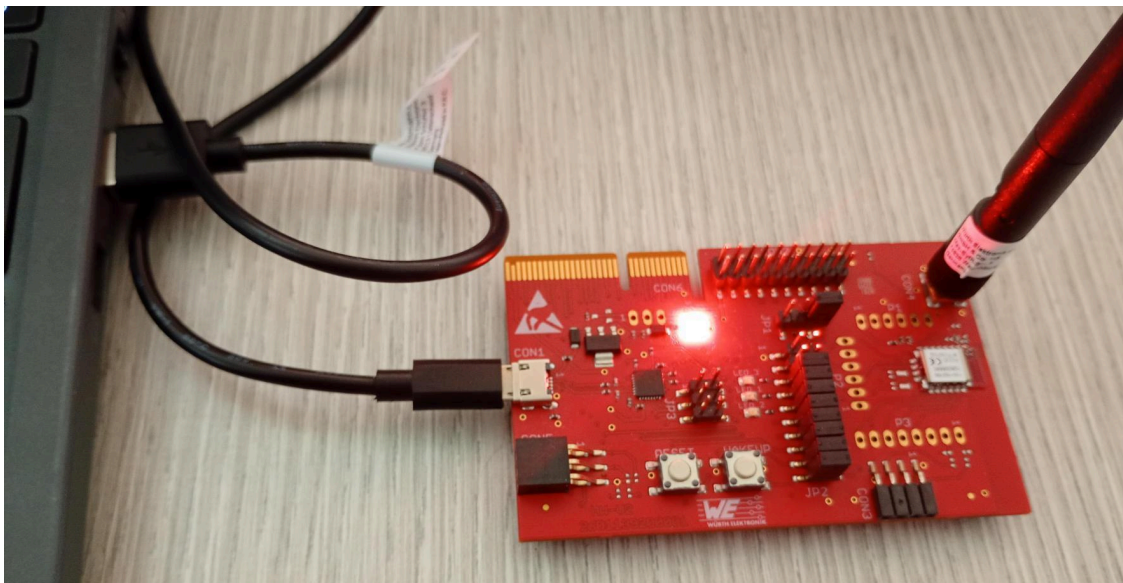


Fig. 17 Collegamento della scheda Thyone-I con antenna esterna durante la fase di test preliminari sul campo, utilizzato un normale connettore USB a micro USB.

2.2.3 I dati ottenuti

I dati ottenuti indicano che la tecnologia a 2.4 Ghz utilizzata da questa scheda funziona **meglio su brevi distanze**, inferiori ad 80 m.

Si sono verificati **alcuni risultati inattesi**. Per esempio, il profilo RF 2 (1000 kbit/s) ha dato in alcuni casi risultati migliori del profilo RF 0 (128 kbit/s), anche se in teoria una velocità più bassa dovrebbe essere più affidabile. Questo può essere dovuto ai disturbi presenti nella banda a 2.4 GHz, che è molto usata da tanti dispositivi (Wi-Fi, Bluetooth, ecc.). Per ridurre il problema, in futuro si potrebbe provare ad usare un canale di trasmissione meno affollato.

Di seguito sono mostrati i **risultati dei test** in due tabelle: la prima ([Tab. 1](#)) riguarda l'**intensità del segnale (RSSI)** mentre la seconda ([Tab. 2](#)) mostra la percentuale di pacchetti persi.

Nella Tabella 1, il valore RSSI è sempre negativo. Più è vicino allo zero (es. -82 dBm), maggiore sarà l'intensità del segnale ricevuto. Più è lontano (es. -97 dBm), più il segnale è debole. Dove c'è una "x", il segnale era troppo debole e la comunicazione non è riuscita.

Nella Tabella 2, la percentuale indica quanti pacchetti sono andati persi. Più è alta, peggiore è stata la comunicazione (0% = tutti ricevuti, 70% = la maggior parte persi).

In generale, quando il segnale è debole (RSSI molto negativo) si perdono più pacchetti, come previsto.

Intensità segnale RSSI [dBm]

Profilo RF	Test in edificio con ostacoli		Distanza al parco senza ostacoli		
	10 m	25 m	80 m	50 m	80 m
RF 0, ANT Int → Est	-82	-91	-91	-92	-96
RF 2, ANT Int → Est	x	x	-87	x	x
RF 0, ANT Est → Int	-88	-92	-93	-95	-97
RF 2, ANT Est → Int	-88	-90	-86	x	x
RF 0, ANT Int → Int	-95	x	-91	x	x
RF 2, ANT Int → Int	x	x	-87	x	x

Tab. 1 Intensità del segnale ricevuto (RSSI) in dBm per diverse configurazioni antenna-profilo RF, misurata in un edificio con ostacoli (Fig. 14) e in un parco senza ostacoli (Fig. 15). I profili sono RF 0 (128 kbit/s) e RF 2 (1000 kbit/s). Le "x" indicano assenza di ricezione.

Percentuale pacchetti non ricevuti [%]

Profilo RF	Test in edificio con ostacoli		Distanza al parco senza ostacoli		
	10 m	25 m	80 m	50 m	80 m
RF 0, ANT Int → Est	5,16	x	x	x	x
RF 2, ANT Int → Est	x	x	37,5	x	x
RF 0, ANT Est → Int	5,16	42,11	x	x	23,81
RF 2, ANT Est → Int	9,52	32	11,11	x	x
RF 0, ANT Int → Int	70	x	x	x	x
RF 2, ANT Int → Int	x	x	28,57	x	x

Tab. 2 Percentuale di pacchetti non ricevuti [%] per diverse configurazioni antenna-profilo RF, misurata in un edificio con ostacoli (Fig. 14) e in un parco senza ostacoli (Fig. 15). I profili sono RF 0 (128 kbit/s) e RF 2 (1000 kbit/s). Le "x" indicano assenza di ricezione.

2.3 Scrittura del codice per test più approfonditi

Dopo aver compreso i reali limiti di queste schede grazie ai test effettuati, ho cominciato a sviluppare un'**integrazione a basso livello** della scheda Thyone-I per il **microcontrollore STM32** per eseguire dei test più approfonditi.

Una libreria in questo caso funge da "ponte" tra l'architettura hardware della scheda Thyone-I con tutte le sue relative configurazioni e il microcontrollore che andremo ad utilizzare. Dunque i comandi che abbiamo visto in precedenza ([Par. 2.1.2](#)) sono stati tradotti per essere eseguiti su un hardware diverso ed in una forma differente rispetto al WE UART Terminal o al programma Python.

Il microcontrollore che andremo ad utilizzare nello specifico è il **NUCLEO-U545RE-Q** ([Fig. 18](#)) con chip **STM32** ed **architettura ARM a 32 bit**.

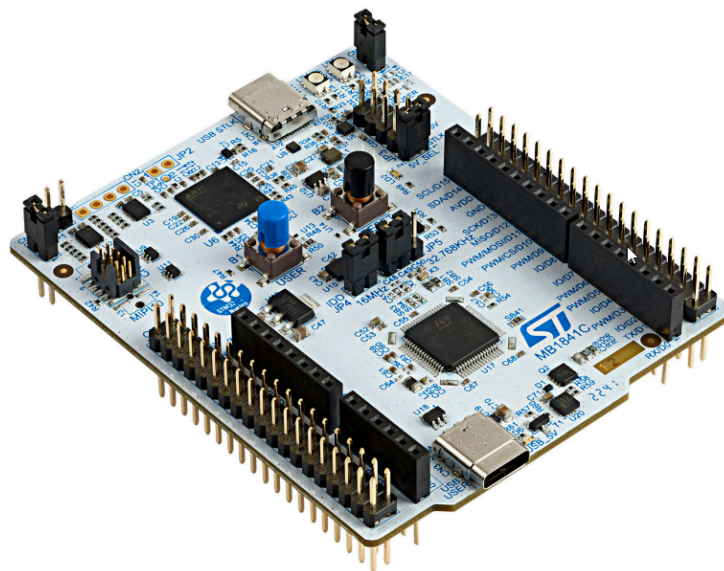


Fig. 18 Nell'immagine di riferimento la scheda NUCLEO-U545RE-Q con chip STM32 ed architettura ARM a 32 bit.

2.3.1 Flusso del programma

Prima di cominciare a scrivere l'integrazione a basso livello è importante avere bene chiaro **il flusso di lavoro del programma** che vogliamo realizzare e dei **principali parametri da configurare**.

I **diagrammi di flusso** mostrati saranno suddivisi in **tre categorie principali** in base al loro funzionamento:

- **SETUP**: L'invio di comandi specifici di **configurazione** tra quelli descritti in precedenza ([Par. 2.1.2](#)), in genere viene eseguito all'avvio di un programma per impostare correttamente tutti i **parametri della scheda**. Oppure anche durante l'esecuzione normale del programma, ad esempio per far entrare la scheda Thyone-I in una condizione di sleep. ([Fig. 19](#))

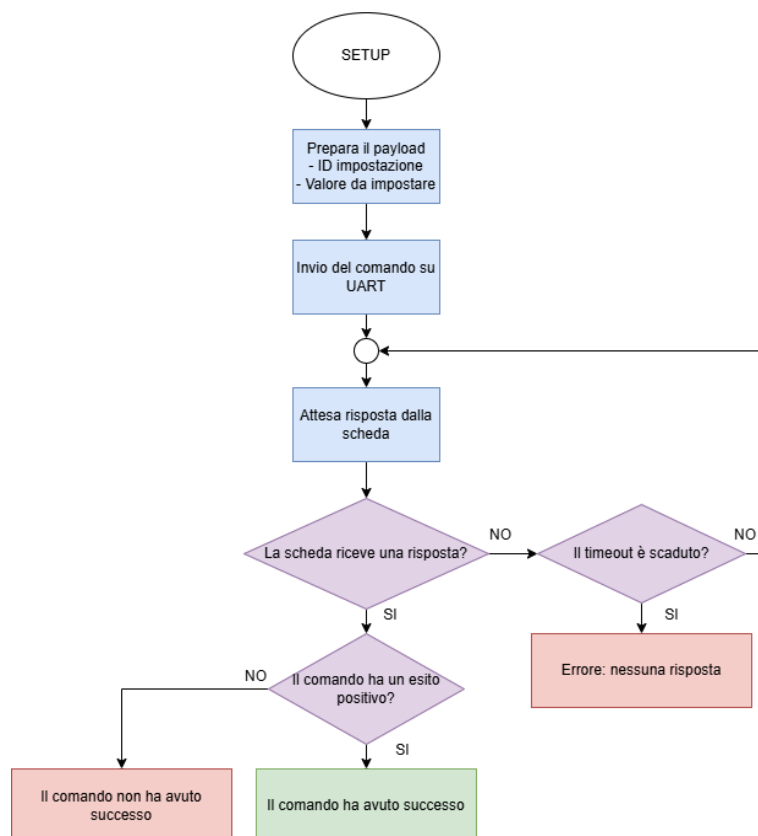


Fig. 19 Diagramma di flusso della funzione di *SETUP* del programma per l'integrazione a basso livello della scheda Thyone-I.

- **BROADCAST:** La creazione del comando di broadcast, secondo le regole di struttura dei dati precedentemente illustrate ([Par. 2.1.3](#)). **Il pacchetto viene assemblato** in tutte le sue parti, **viene calcolata la sua lunghezza e calcolato il checksum** di verifica del dato. ([Fig. 20](#))

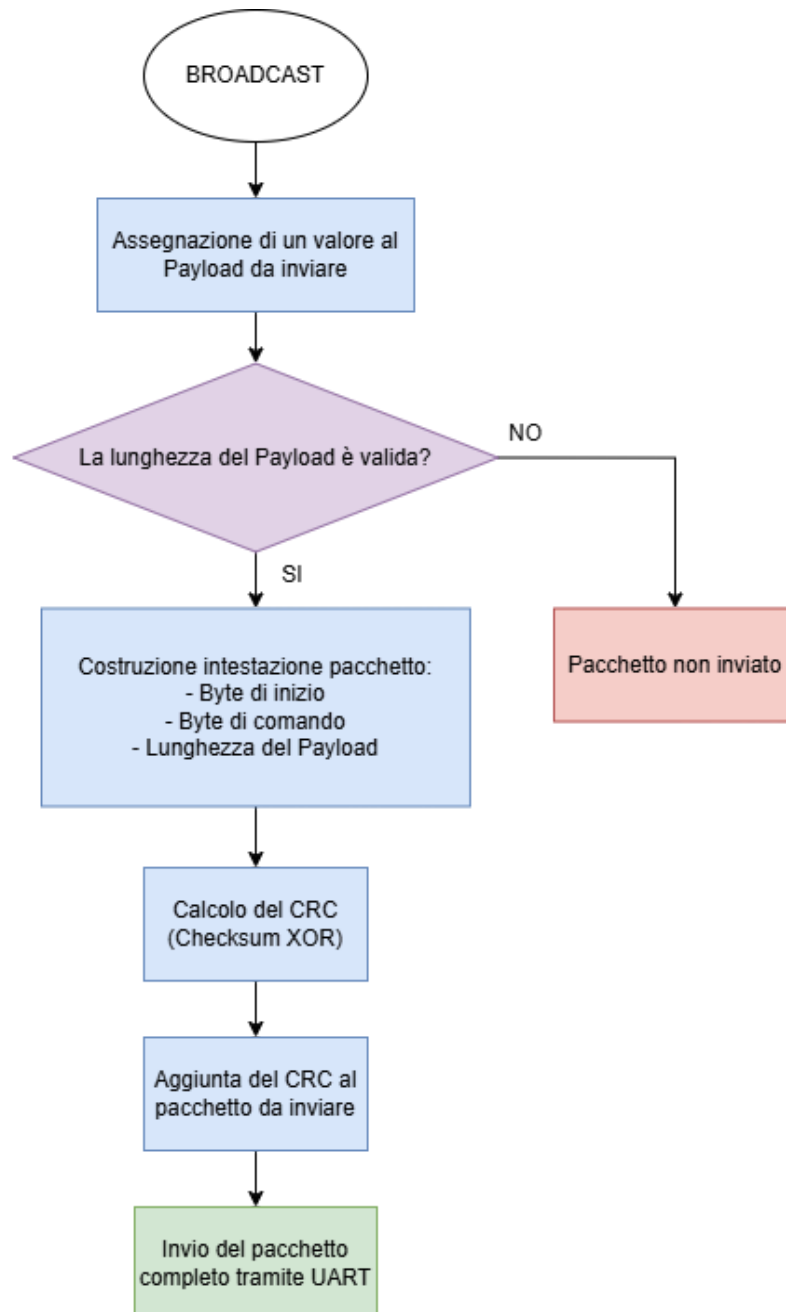


Fig. 20 Diagramma di flusso della funzione di BROADCAST del programma per l'integrazione a basso livello della scheda Thyone-I.

- **RECEIVE: L'estrapolazione dei dati ricevuti via radio** dalla struttura dei dati ([Par. 2.1.3](#)). Fa uso di una **funzione di interrupt** per individuare subito l'arrivo dei dati anche durante la normale esecuzione del programma. Dal pacchetto si ottiene il checksum ricevuto e si confronta con il valore calcolato sui dati ricevuti per **verificarne l'integrità**. Si ricava inoltre il **valore di RSSI**, l'intensità del segnale ricevuto ([Fig. 21](#)).

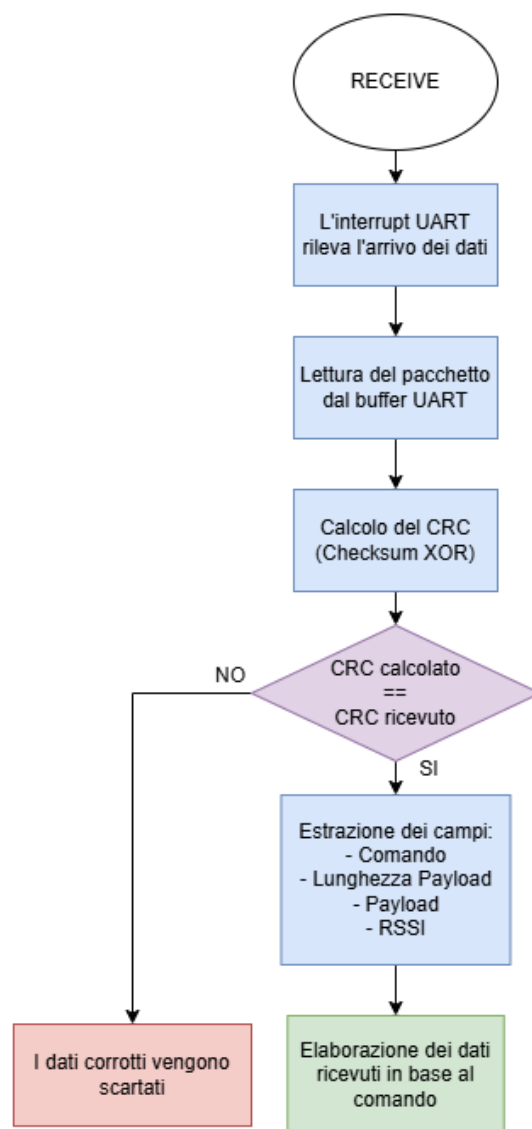


Fig. 21 Diagramma di flusso della funzione RECEIVE del programma per l'integrazione a basso livello della scheda Thyone-I.

2.3.2 Configurazione del microcontrollore

Per configurare il microcontrollore STM32 la **STMicroelectronics**, l'azienda produttrice di questi chip, ci fornisce il **software STM32CubeMX** che è un **generatore di codice** per la configurazione dei parametri su queste schede elettroniche ([Fig. 22](#)).

Le impostazioni che sono state modificate riguardano:

- L'attivazione di una **comunicazione seriale UART** su due pin dedicati della scheda **TX** e **RX** che rispettivamente gestiranno l'invio e la ricezione dei dati con la scheda Thyone-I ([Fig. 22](#)).

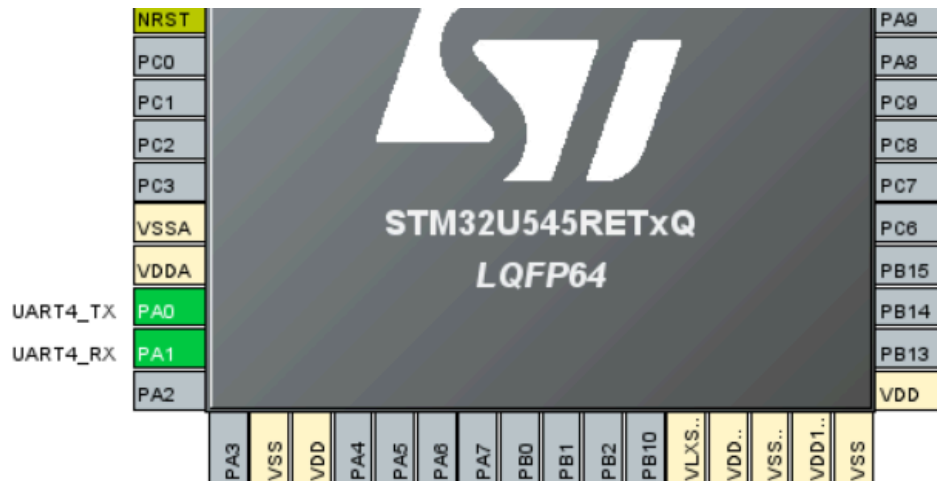


Fig. 22 Configurazione dei pin GPIO del microcontrollore STM32U545RETxQ all'interno del software STM32CubeMX. Vengono abilitati i pin PA0 E PA1 e configurati come TX ed RX per la comunicazione UART.

- La **selezione della velocità di trasmissione seriale** dei dati, in questa situazione utilizzeremo la massima velocità disponibile della scheda Thyone-I (1'000'000 baud) assicurandoci che sia supportata anche dal microcontrollore STM32. Bisogna inoltre assicurarsi che la frequenza di clock dedicata alla seriale consenta tale velocità di trasmissione.

- **L'utilizzo di un interrupt per la ricezione** dei dati: un interrupt è un meccanismo gestito lato hardware che permette al microcontrollore di sospendere temporaneamente il processo in corso per gestire un **evento immediato**, in questo caso l'arrivo dei dati su seriale UART.

A modifiche concluse il software STM32CubeMX genererà automaticamente parte del codice che gestisce questi registri.

2.3.3 Collegamento tra le schede

È necessario collegare tra di loro la scheda Thyone-I di Wurth e la scheda NUCLEO-U545RE-Q basata su STM32 ([Fig. 23, 24](#)).

La tecnologia UART fa uso di due pin per la comunicazione:

- **TX, per inviare un dato**
- **RX, per ricevere un dato**

e la connessione a massa (**GND**) comune.

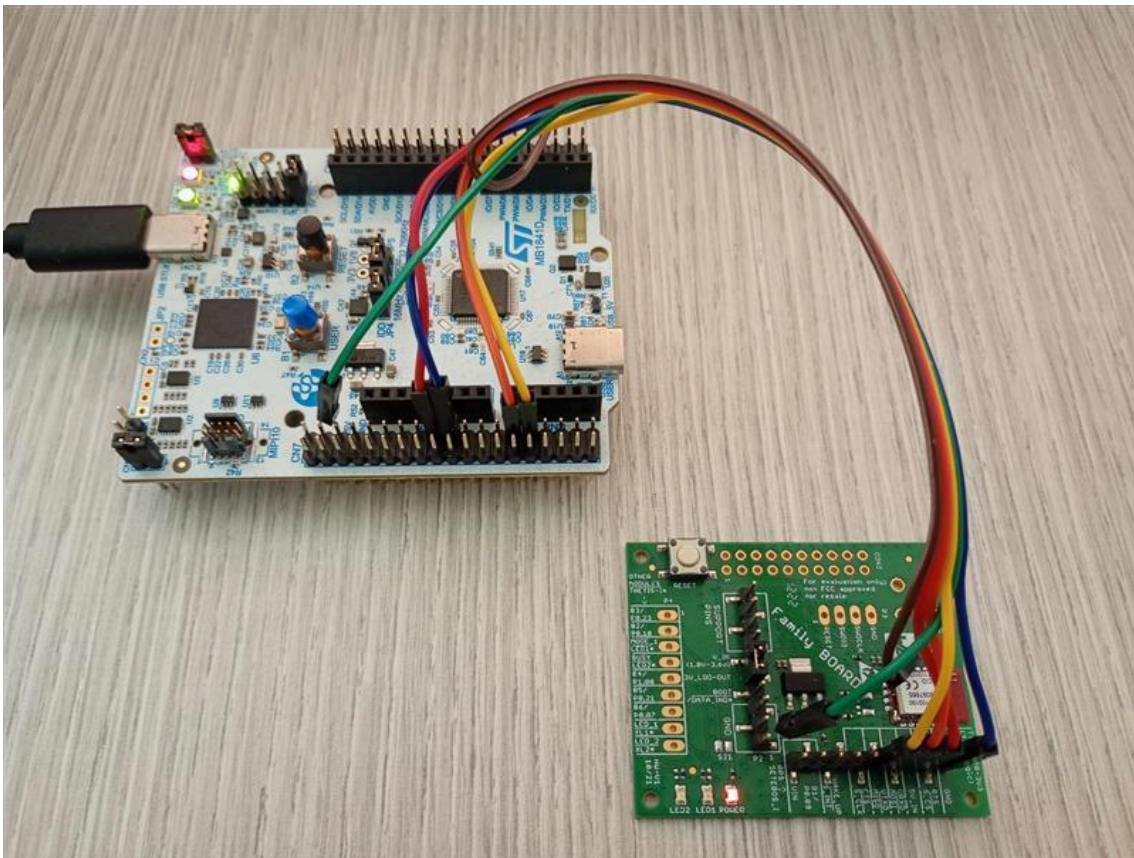


Fig. 23 Immagine che illustra il collegamento tra la scheda NUCLEO-U545RE-Q STM32 e la scheda Thyone-I (antenna interna) per effettuare una comunicazione seriale UART.

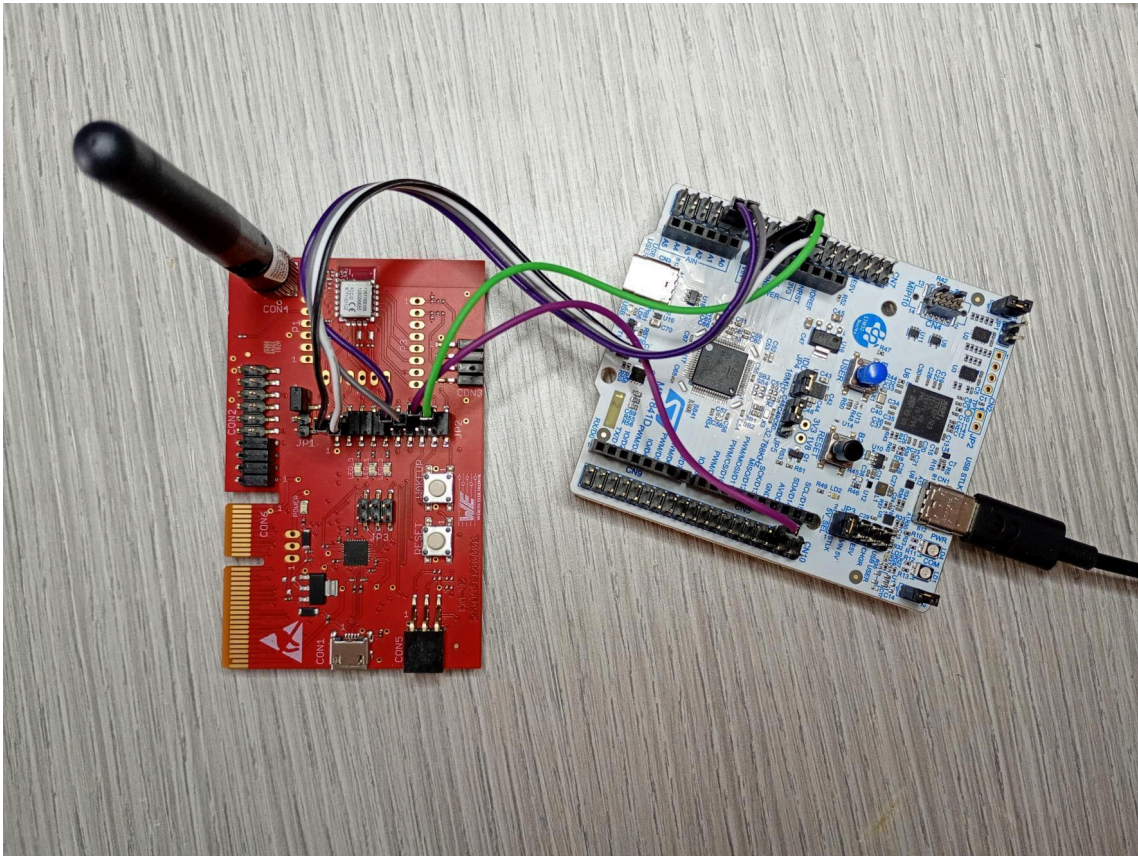


Fig. 24 Immagine che illustra il collegamento tra la scheda NUCLEO-U545RE-Q STM32 e la scheda Thyone-I (antenna esterna) per effettuare una comunicazione seriale UART.

2.3.4 Programmazione secondo le linee guida

Per programmare la scheda microcontrollore si installano i **driver per l'ST-LINK** che è il chip hardware integrato all'interno della scheda NUCLEO-U545RE-Q, incaricato di sovrascrivere il firmware dell'STM32 e si utilizza il programma **STM32CubeIDE** come editor di testo per scrivere il codice.

A questo punto si può trasferire la logica del programma all'interno del codice, questo codice è stato scritto in **linguaggio C**, particolarmente adatto al settore embedded. Il C, in quanto linguaggio a basso livello, garantisce un'ottima integrazione con l'hardware ed efficienza in termini di prestazioni e di energia consumata.

Le linee guida da seguire per programmare sono diverse, alcune devono essere seguite obbligatoriamente mentre altre sono consigliate.

Il **MISRA C** è un documento fondamentale per la programmazione in C che le illustra e che se vengono seguite garantisce una migliore affidabilità per la **sicurezza** del firmware rilasciato.

Ad esempio, è importante che il codice, quando viene compilato non generi degli avvisi, quando succede se non si può correggere è necessario spiegare perché.

Oppure se una variabile ha visibilità sull'intero codice non può avere lo stesso nome di una variabile che si trova in un blocco di codice più piccolo e che quindi ha visibilità solo al suo interno, questo perché si rischia di fare confusione tra le due ([Fig. 25](#)).

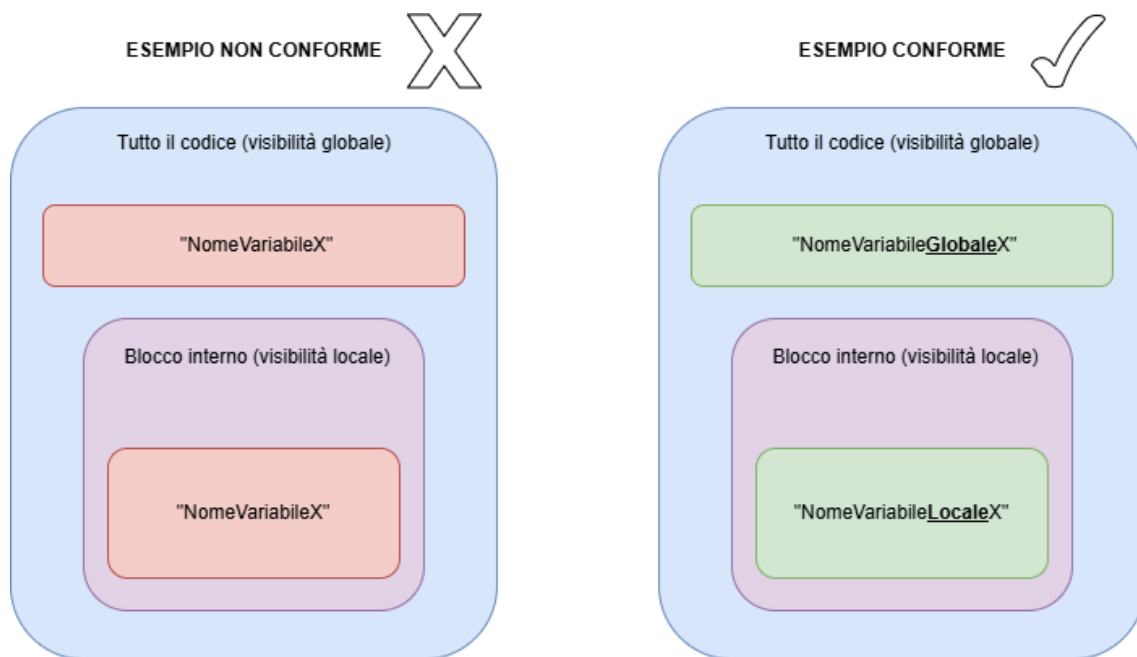


Fig. 25 Rappresentazione grafica di un esempio conforme di una regola MISRA C da seguire durante la scrittura del codice di programmazione. Una variabile locale non può avere lo stesso nome di una variabile globale, perché potrebbe creare confusione.

Fondamentale è anche mantenere il **codice sempre documentato** attraverso dei commenti all'interno di esso.

2.3.5 Modalità di test

Durante le prove, il microcontrollore STM32 ha inviato 1 MB di dati attraverso la scheda Thyone-I, sfruttando la massima velocità disponibile sulla porta UART, pari a 1.000.000 di baud.

Per la trasmissione radio abbiamo messo a confronto due diversi profili: il profilo 0 a 128 kbit/s, pensato per comunicazioni a lungo raggio grazie alla sua bassa velocità, e il profilo 1 a 500 kbit/s, che rappresenta una soluzione a velocità medio-bassa. Abbiamo scelto il profilo 1 perché rappresenta un compromesso intermedio tra velocità e affidabilità, dopo aver constatato nei primi test che il profilo 2 risultava eccessivamente sensibile.

Il flusso dei dati è stato testato in una sola direzione, dalla scheda dotata di antenna interna verso la scheda con antenna esterna. Le prove si sono svolte in due ambienti molto diversi tra loro: il tetto dell'edificio aziendale, dove il segnale ha dovuto attraversare diversi ostacoli in cemento spostandosi tra varie posizioni ([Fig. 26](#)), e il Parco Paradiso a Torino, un'area completamente aperta e priva di ostacoli, dove abbiamo potuto testare la comunicazione a distanze differenti ([Fig. 27](#)).



Fig. 26 L'immagine di riferimento rappresenta il tetto dell'edificio e le diverse posizioni in cui sono stati eseguiti i test. Il quadrato rosso indica la scheda con antenna esterna, mentre i quadrati arancioni indicano la scheda con antenna interna con le relative distanze dalla scheda con antenna esterna.

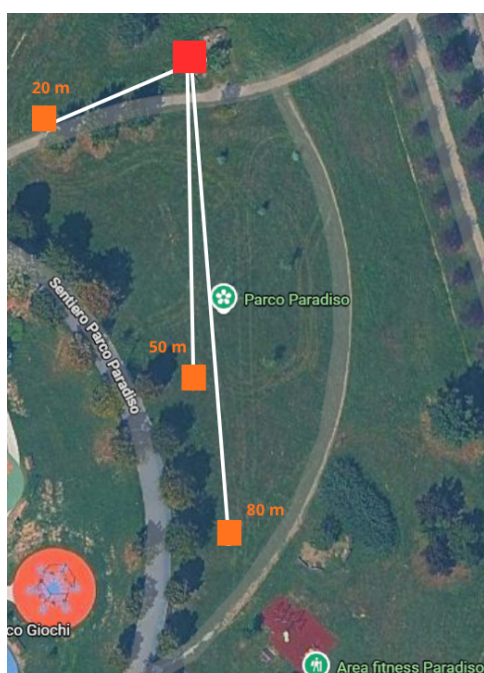


Fig. 27 L'immagine di riferimento rappresenta le diverse posizioni in cui sono stati eseguiti i test al Parco Paradiso a Torino. I quadrati rossi indicano la scheda con antenna esterna, mentre i quadrati arancioni indicano la scheda con antenna interna con le relative distanze dalla scheda con antenna esterna.

2.3.6 I dati ottenuti

I risultati dei nuovi test mostrano che il profilo 0 a 128 kbit/s offre **prestazioni complessivamente accettabili**, con una percentuale di pacchetti persi inferiore al 10% fino a 50 metri in campo aperto e fino a 15 metri in presenza di ostacoli. Il profilo 1 a 500 kbit/s, al contrario, si è rivelato inaffidabile quando il segnale doveva attraversare ostacoli in cemento, con perdite elevate e blackout totali della comunicazione; in campo aperto, invece, le sue prestazioni sono risultate quasi comparabili a quelle del profilo 0 fino a 50 metri.

La tecnologia a 2,4 GHz della scheda Thyone-I è quindi adatta per applicazioni su **distanze limitate**, preferibilmente entro i 50 metri, mentre per coperture maggiori si consiglia l'adozione di soluzioni alternative. Di seguito i dati ottenuti indicati come percentuale di pacchetti non ricevuti ([Tab. 3](#)).

Percentuale pacchetti non ricevuti [%]

Profilo RF	Test su tetto con ostacoli			Distanza al parco senza ostacoli		
	11 m	15 m	16 m	25 m	50 m	80 m
RF 0: 128 kbit/s	6,53	5,16	40,29	0	3,68	26,33
RF 1: 500 kbit/s	x	42,38	x	0,33	5,66	44,71

Tab. 3 Percentuale pacchetti non ricevuti (%) per due profili RF, misurata in due scenari: su un tetto con ostacoli e in un parco libero da ostacoli.

I profili corrispondono a diverse velocità di trasmissione: RF 0 (128 kbit/s) e RF 1 (500 kbit/s). Le distanze indicate nella prima riga si riferiscono, per i primi tre valori (11 m, 15 m, 16 m), alla prova su tetto con ostacoli ([Fig. 26](#)); per gli ultimi tre (25 m, 50 m, 80 m), alla distanza nel parco senza ostacoli ([Fig. 27](#)). Il simbolo "x" indica che in quella condizione non è stato ricevuto alcun dato.

3. Riflessioni finali e conclusioni

Questo progetto mi ha permesso di acquisire competenze concrete, dalla lettura e comprensione di un datasheet, all'esecuzione di test sul campo, fino alla scrittura di codice C conforme a standard industriali. Ho imparato a gestire la comunicazione UART a basso livello, a configurare periferiche su STM32 e a sviluppare una logica di trasmissione e ricezione con la verifica di integrità dei dati. Inoltre, ho compreso l'importanza di documentare ogni fase del lavoro, sia per uso interno aziendale sia per futuri sviluppatori che utilizzeranno la mia libreria.

In conclusione, questa è stata per me un'**esperienza di grande valore**, che mi ha avvicinato alle **reali dinamiche di sviluppo nel settore embedded**.

4. Glossario

2.4 GHz: Banda ISM (2,400 - 2,4835 GHz) usata da ricetrasmittitori come Thyone-I. Offre alta velocità (fino a 1 Mbit/s) ma è sensibile a ostacoli e interferenze.

ARM: Advanced RISC Machine. Famiglia di architetture per microprocessori RISC a 32 e 64 bit, ampiamente utilizzata in sistemi embedded per la loro efficienza energetica.

Baud Rate: Velocità di trasmissione dati in una comunicazione seriale (come UART). Indica il numero di variazioni di segnale (simboli) al secondo. Nell'ambito di questo progetto, un Baud Rate di 1'000'000 baud equivale a una velocità di trasmissione di circa 1 Mbit/s.

Bit: Unità fondamentale dell'informazione digitale, che può assumere il valore 0 o 1. Otto bit costituiscono un byte. Nel progetto, la velocità di trasmissione viene espressa in kbit/s (kilobit al secondo).

Byte: Unità di misura dell'informazione digitale composta da 8 bit. Nel progetto viene utilizzata per indicare la dimensione dei dati trasmessi, come il payload massimo di 224 byte della scheda Thyone-I.

Checksum (CS): Valore calcolato a partire da un blocco di dati digitali allo scopo di rilevare errori durante la trasmissione o la memorizzazione. In questo progetto si utilizza un calcolo XOR sui byte del pacchetto.

Flow Control (RTS/CTS): Controllo di flusso hardware. Meccanismo utilizzato nella comunicazione seriale per prevenire la perdita di dati quando un dispositivo non è pronto a ricevere. RTS (Ready To Send) e CTS (Clear To Send) sono i pin dedicati a questa funzione.

Firmware: Software di basso livello programmato all'interno di un dispositivo hardware, come un microcontrollore. Gestisce direttamente l'hardware del dispositivo.

GPIO: (General Purpose Input/Output). Pin di un microcontrollore che possono essere configurati via software per funzionare come ingresso digitale (lettura di un segnale) o uscita digitale (controllo di un segnale).

Interrupt: Segnale hardware o software inviato al processore per indicare un evento che richiede attenzione immediata. Nel progetto, l'interrupt UART permette di gestire i dati in arrivo dalla scheda radio senza dover attendere attivamente in un ciclo infinito.

MAC Address: (Media Access Control address). Indirizzo univoco di 48 bit assegnato a ogni interfaccia di rete. Viene utilizzato per identificare univocamente un dispositivo all'interno di una rete.

Mesh Network: Topologia di rete in cui i nodi sono interconnessi tra loro. Se un nodo non può comunicare direttamente con il gateway, i dati vengono ripetuti dai nodi vicini, aumentando l'affidabilità e la copertura del segnale.

MISRA C: (Motor Industry Software Reliability Association). Insieme di linee guida e regole per lo sviluppo software in linguaggio C, nate per il settore automotive ma diventate standard di riferimento per la programmazione embedded safety-critical.

Payload: La parte di dati trasmessi che rappresenta l'effettivo messaggio o contenuto informativo. Nel caso della Thyone-I, il payload massimo è di 224 byte.

PCB: (Printed Circuit Board). Scheda a circuito stampato. Nel testo, l'antenna PCB indica un'antenna integrata direttamente nelle tracce di rame del circuito stampato, senza componenti esterni sporgenti.

RSSI: (Received Signal Strength Indicator). Indicatore numerico (espresso in dBm) della potenza o intensità del segnale radio ricevuto da un dispositivo. Valori più alti (meno negativi, es: -40 dBm) indicano un segnale migliore e più forte rispetto a valori più bassi (es: -90 dBm).

STM32: Famiglia di microcontrollori a 32 bit basati su core ARM Cortex-M, prodotti da STMicroelectronics.

Timestamp: Sequenza di caratteri che rappresentano una data e un orario preciso (es: 14:35:22) utilizzata per registrare l'istante in cui si verifica un evento.

UART: (Universal Asynchronous Receiver-Transmitter). Protocollo di comunicazione seriale asincrona che utilizza generalmente due linee

(TX per trasmettere ed RX per ricevere) per lo scambio di dati tra dispositivi.

XOR: Exclusive OR. Operatore logico booleano che restituisce VERO (1) solo se gli ingressi sono diversi tra loro. Utilizzato come metodo semplice ed efficace per il calcolo del Checksum.

5. Bibliografia e sitografia

STM32 U545REQ-D: Documentazione

- https://www.st.com/resource/en/data_brief/nucleo-u545re-q.pdf
- https://www.st.com/resource/en/schematic_pack/mb1841-u545req-d01-schematic.pdf
- https://www.st.com/resource/en/technical_note/tn1235-overview-of-stlink-derivatives-stmicroelectronics.pdf

THYONE-I: Documentazione

- <https://www.we-online.com/en/products/components/service/wireless-connectivity-sensors>
- [https://www.we-online.com/components/products/manual/UM_Family-EV-board_26xxx4902x0x1%20\(rev1.6\).pdf](https://www.we-online.com/components/products/manual/UM_Family-EV-board_26xxx4902x0x1%20(rev1.6).pdf)
- <https://www.we-online.com/en/components/products/THYONE-I>
- [https://www.we-online.com/de/components/products/manual/UM_Thyone-I_2611011021000%20\(rev1\).pdf](https://www.we-online.com/de/components/products/manual/UM_Thyone-I_2611011021000%20(rev1).pdf)
- <https://docs.rs-online.com/0b4a/A700000007100849.pdf>

TTL-232R: Documentazione

- https://www.ftdichip.com/Support/Documents/DataSheets/Cables/DS_TTL-232R_CABLES.pdf

WE UART Terminal: Documentazione

- https://symmetron.ua/image/catalog/2025/data_delphi_usermanual.pdf